

A Low Latency, High Frequency Trading System

An end-to-end HFT pipeline on a Xilinx Alveo U55C FPGA

Brendon Chang Tristan Lee Yahir Salas
Mustahsin Zarif Philip Pincencia Marc Boudames

Triton Quantitative Trading • University of California, San Diego

FPGA & High Frequency Trading

- High frequency trading (HFT) systems process millions of messages per symbol and must react in microseconds.
- Decode the market feed, make a decision, and send an order back.

Why an FPGA?

- Reprogrammable hardware that gives deterministic latency
- Parallel: can run every pipeline stage at once

Target: Xilinx Alveo U55C • Virtex UltraScale+ •
100 GbE



Xilinx Alveo U55C accelerator card

Data & Assumptions

- SPY: SPDR S&P 500 ETF (a basket of the 500 names)
- XNAS: Nasdaq equity trade prints (ITCH 5.0)
- OPRA: consolidated options trade prints

Key assumption

- Options traders are more informed than equity traders
- Treat their order flow as information when trading SPY

Design goal: optimize for speed, not margin.

Example feed rows (databento export):

XNAS equity

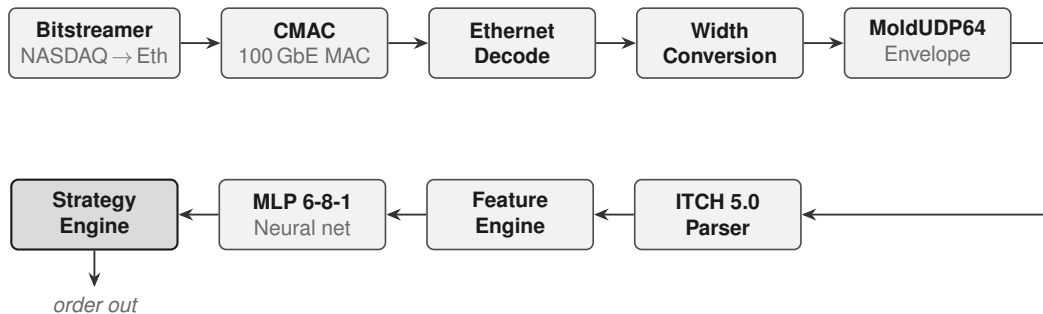
ts_event	side	price	size
09:00:00.334	A	594.86	5
09:00:27.154	A	595.00	31

OPRA options

symbol	side	price	size
SPY 250117C615	N	1.23	1
SPY 241224P589	N	0.04	3

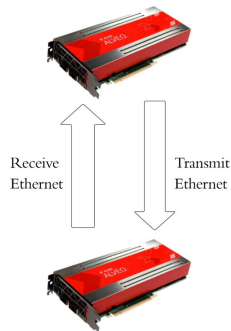
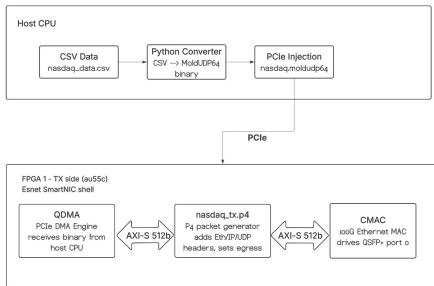
Two synchronized feeds, matched per trading day.

Pipeline Overview



Bitstreamer | Feed Injection

Packet Bit Streamer



AXI-Stream Packet Transport

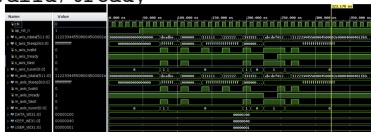
Objective

Provide high-throughput packet transport between network and parser stages using AXI-Stream interfaces.

Responsibilities

- Receive packet streams from upstream modules
- Transport packet payloads using 512-bit AXI-Stream
- Preserve packet boundaries (`tlast`)
- Preserve byte-valid information (`tkeep`)
- Preserve packet metadata (`tuser`)
- Support flow control through `tvalid`/`tready`

Signal	Purpose
<code>tdata</code>	Packet payload
<code>tvalid</code>	Data available
<code>tready</code>	Receiver ready
<code>tlast</code>	End of packet
<code>tkeep</code>	Valid bytes
<code>tuser</code>	Metadata



Simulation result: packet payloads and control signals propagate correctly through the AXI-Stream datapath under normal operation and backpressure.

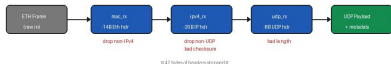
MAC → IPv4 → UDP

Objective

Strip 42 bytes of protocol headers (Ethernet + IPv4 + UDP) from incoming market data packets and deliver validated UDP payloads with metadata to downstream ITCH decoders.

Responsibilities

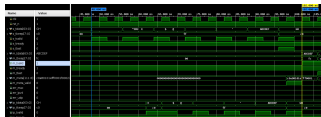
- `mac_rx`: strip 14B Ethernet header, filter on EtherType 0x0800 (IPv4)
- `ipv4_rx`: strip 20B IP header, verify checksum, filter Protocol = UDP (0x11)
- `udp_rx`: strip 8B UDP header, output payload + 112-bit metadata sideband
- 64-bit AXI-Stream bus (`tvalid`/`tready`/`tdata`/`tkeep`/`tlast`) throughout
- 14-cycle pipeline latency at 250 MHz = 56 ns end-to-end



Output

Description

<code>m_axis_t*</code>	UDP payload AXI-Stream
<code>src_ip[31:0]</code>	Source IPv4 address
<code>dst_ip[31:0]</code>	Destination IPv4 address
<code>src/dst_port</code>	UDP port numbers
<code>udp_len[15:0]</code>	UDP payload length



Simulation result: all 3 tests passed: valid IPv4/UDP payload decoded correctly with matching metadata; non-IPv4 frames dropped at `mac_rx`, TCP dropped at `ipv4_rx`.

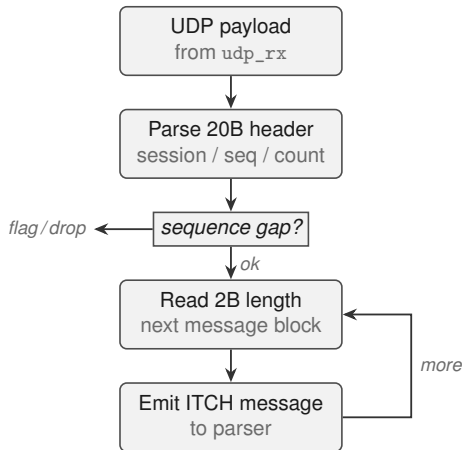
MoldUDP64 Envelope Parser

Objective

Unwrap the NASDAQ MoldUDP64 transport envelope and emit the individual ITCH 5.0 messages it carries.

Responsibilities

- Strip the 20-byte header (10B session, 8B sequence, 2B count)
- Track the sequence number for gap detection
- Walk each message block: read its 2-byte length prefix, then forward exactly that many payload bytes
- Emit one clean ITCH message per block downstream



ITCH 5.0 Parser & Limit Order Book

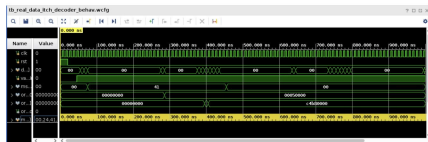
Responsibilities

- Receive 512-bit AXI-Stream data from the 100G CMAc interface
- Deserialize to 8-bit byte stream, respecting tkeep for partial frames
- Parse all four ITCH 5.0 message types: Add Order, Execute, Cancel, Delete
- Extract price, shares, and side from each message at 1 byte/cycle
- Update Limit Order Book hardware registers after every message
- Output best_bid_price[31:0] and best_ask_price[31:0] continuously

Signal	Purpose
tdata	512-bit packet payload
tkeep	Valid byte lanes mask
tvalid	Data available
tready	Receiver ready
tlast	End of frame
byte_out	8-bit deserialized byte
best_bid	Best bid price [31:0]
best_ask	Best ask price [31:0]

MODULE HIERARCHY

```
top_parser_tx_10b
├── axi3_312_tx_8_adapter
└── 110n_decoder
    └── order_book
```



<840 ns

END-TO-END LATENCY

100 MHz

CLOCK SPEED

8.71 ns

TIMING SLACK • U55C

<0.01%

CHIP UTILIZATION

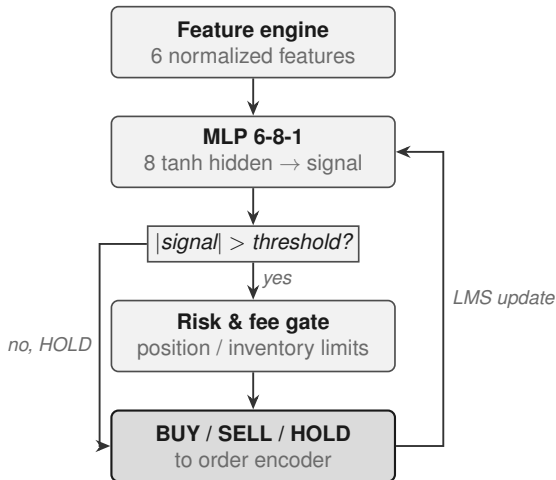
Strategy Engine

From features to an order

- 6 features from both order books:
momentum, vol-imbalance, call/put-imbalance,
premium-bias, volatility-skew, inventory-z
- 6-8-1 MLP: 6 inputs $\rightarrow$ 8 hidden nodes $\rightarrow$
one signal
- Threshold + risk / position limits + fees
- Online optimizer retrains the weights live
- Emit BUY / SELL / HOLD order

Example trades:

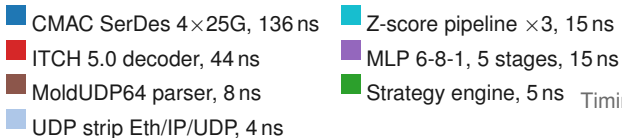
cyc	side	price	pos	pnl
55	SELL	595.01	-17	0
63	BUY	595.21	-72	-2.76
73	SELL	595.20	-112	-41.96



Results

Simulation | wire-to-order latency
decomposition

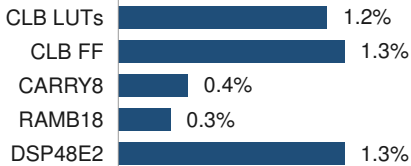
clk_net = 200 MHz $\rightarrow$ 5 ns per cycle



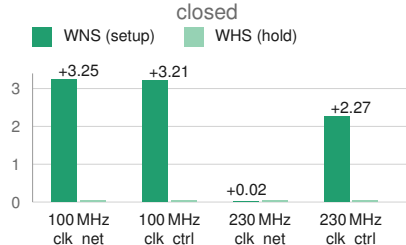
Total wire to order $\approx$ **227 ns** ($\approx 0.227 \mu\text{s}$),
bit-identical to the C++ reference, 0 errors

Synthesis | Alveo U55C

Resource utilization (% of total U55C capacity)



Timing margins: WNS / WHS (ns), all positive $\Rightarrow$ timing



Setbacks & Future Work

Setbacks, and how we closed them

- ITCH parser syntax errors in early sim (→ fixed, all four message types now parse)
- JTAG connectors were not up (and couldn't get them up) so we switched to use SMART NIC to synthesize as a temporary solution

Future Work

- JTAG connectors were not available on the card, which blocked on card validation
- Admins were slow to help with hardware access
- Would like to run the full implementation on card in the future

Thank You

Questions?

Recruitment

Future Recruitment

- Come talk to us after class